

AegisAI: An Agentic AI Policy Enforcement Gateway for Customer-Support Systems

Divya M*

Assistant Professor

Computer Science and Engineering
Rajalakshmi Engineering College Chennai,
India

divya.m@rajalakshmi.edu.in

Ishitha S

Student

Computer Science and Engineering
Rajalakshmi Engineering College Chennai,
India

230701117@rajalakshmi.edu.in

Hemashri U

Student

Computer Science and Engineering
Rajalakshmi Engineering College Chennai,
India

230701114@rajalakshmi.edu.in

Ishwari Rajmohan

Student

Computer Science and Engineering
Rajalakshmi Engineering College Chennai, India

230701118@rajalakshmi.edu.in

Jaya Bharathi M

Student

Computer Science and Engineering
Rajalakshmi Engineering College Chennai, India

230701126@rajalakshmi.edu.in

Abstract -AI-powered customer-support agents can improve response times, but may also represent a business risk by promising unauthorized refunds, leaking sensitive data, or performing unsafe actions. Most existing guardrails focus on content safety and post-hoc monitoring, rather than preventing unsafe actions from reaching customers in the first place. We present AgeisAI, an agentic AI system with a multi-agent architecture that enforces customer-service policies before responding or executing business tools. AgeisAI places a Policy Enforcement Gateway between a support agent and an operations agent, measures the actions suggested according to deterministic business rules, and employs a four-stage decision-making process (allow, modify, block, escalate). The system provides an strict authorization boundary that checks the tool names and their arguments before action execution and keeps a unified audit trail from the customer input to the result. In our pilot evaluation on 30-50 labelled test cases involving refunds, verification, delivery promises, and escalation, AgeisAI denies the actions not allowed, produces safe alternatives, and ensures full accountability of all actions. Our findings show that pre-action enforcement with inter-agent control is an executable and auditable approach for AI compliance in customer support.

Keywords: Agentic AI, Multi-agent systems, Policy enforcement, Tool authorization, Customer support, Audit trail, AI governance.

I. INTRODUCTION

The rapid deployment of AI-powered customer support agents has transformed sectors such as e-commerce, tourism, financial services and government organizations. These agentic systems allow companies to process high-volume requests, respond promptly and automate routine tasks like issuing refunds and processing cancellations. Nevertheless, deploying these agents carries inherent business, legal and reputational risks. Companies face potential vulnerabilities including unauthorized commitments, unsafe actions and confidential data leaks, if an AI agents operate beyond their authorized boundaries (BBB 2026; ITPro 2026).

Recent real-world incidents reveal these vulnerabilities. An Air Canada chatbot fabricated a non-existent bereavement refund

policy and a tribunal subsequently ruled the airline bound by the bot's promise, awarding damages. Similarly, a Chevrolet dealership's AI assistant was manipulated via prompt injection into selling a \$58,000 vehicle for just \$1. Other notable failures include a BMW Toronto buyback bot offering roughly CAD \$7,000 above market valuation and an Indie Campers assistant providing incorrect trip rules that resulted in a €1,500 loss. Furthermore, chatbots have leaked millions of customer records (Sears), exposed session cookies under prompt injection (Lenovo Lena), insulted their own brand (DPD), and advised businesses to violate labor laws (NYC "MyCity"). Industry surveys say 74% of enterprises have suspended or rolled back AI customer-service deployments due to hallucinations, data leaks and a lack of auditability. Moreover, over 90% of AI-related consumer reviews since 2023 have been negative, frequently highlighting "digital entrapment" in frustrating refund loops.

Therefore, the integration of a system capable of detecting policy violations in customer-support interactions and preventing unsafe actions before they impact users is critical for establishing a reliable and accountable AI-augmented service environment (CSA 2026; NIST 2026). While current AI governance and guardrail efforts rely on content-level filters for toxicity, PII and prompt injection, as well as output validation, refunds, false policy promises, and data exposures continue to occur, leading to lawsuits, regulatory examination and brand damage.

Prior work on AI agent security and governance has explored runtime policy enforcement, tool-call authorization and multi-agent safety frameworks. However, most such solutions currently are generic or rely on post-hoc monitoring rather than robust pre-execution enforcement mechanisms.

To bridge these gaps, we propose AegisAI, a framework that integrates a multi-agent policy enforcement architecture with

deterministic business rules and fine-grained tool-call authorization, aimed at the customer-support domain. The primary objectives are real-time evaluation of proposed AI responses and actions, pre-execution blocking of unauthorized tool calls, the generation of safe alternative responses and the maintenance of a unified audit trail from the initial customer request to the final decision. In conclusion, AegisAI aims to create a more compliant and safer customer-support environment where AI agents can help users without exceeding their authorized boundaries, promoting trust, accountability and operational resilience.

II. LITERATURE REVIEW

The incorporation of autonomous AI agents within organizational processes has driven research into the areas of agent security, runtime governance, policy-as-code and authorization. The existing literature covers four major domains: runtime tool control, policy autoformalization, multi-agent protocol governance and security gateways.

A. Runtime Policy Enforcement and Tool Control

Sigdel and Baral [1] developed a Policy-First mechanism that acts as a mediator for tool calls using permission-based constraints and approval gates. They demonstrated a trade-off where stricter policies increased violation prevention from 0% to 68%, but simultaneously reduced the task success rate from 36% to 7%. However, the approach manages general tool governance and does not verify domain-related customer support business policies or parameters against the authorization envelope. Li et al. [2] proposed VIGIL, a runtime reference monitor that grounds skill specifications into finite-trace SMT constraints to detect temporal violations (at 95.8% recall). While effective for procedural correctness, VIGIL focuses on skill compliance rather than enforcing enterprise business rules or inter-agent authorization.

B. Policy Autoformalization and Formal Verification

Mondl et al. [3] used an LLM generator-critic framework for translating policies into formally verifiable Cedar policies to prevent up to 100% of malicious writes in clinical benchmark tests. Although their solution offers strong formal guarantees, its focus lies solely in the medical context. It fails to address the requirements for a multi-agent architecture with proper argumentation validation and centralized audit logs.

C. Multi-Agent Protocols, Threat Modeling, and Authorization Propagation

Gupta et al. [4] have proposed a detailed threat model covering various attack vectors for single agents, such as prompt injection, memory poisoning, and multi-agent attacks, emphasizing that existing literature neglects runtime security controls. Narajala et al. examined Google’s Agent-to-Agent (A2A) protocol stack [5], uncovering vulnerabilities related to spoofed AgentCards and re-transmitted tasks. They suggested

using mutual TLS certificates and digital signature verification instead. Lastly, [6] Tallam explored authorization propagation within delegations. He argued in favor of cryptographic audit logchains. However, despite these advancements in network and transport protocols, their work does not consider business restrictions.

D. Architectural Gateways and Governance-as-a-Service

In Tran et al.’s [7] approach to agent security via network protocols, an Agent Policy Gateway sidecar intercepts outgoing API requests. Likewise, Gaurav et al. [8] introduced the concept of Governance-as-a-Service (GaaS), which is an additional layer for enforcement of multi-agent compliance. Despite these architectures achieving separation of governance from model weights, they cannot be used to evaluate multi-turn dialogue context, enforce fine-grained parameter envelope limits or remediate blocked responses.

E. Research Gap and Position of AegisAI

While prior literature has established strong foundations for runtime monitoring, protocol security, and policy formalization, it lacks a specialized pre-execution policy gateway tailored to customer support. AegisAI bridges this gap by decoupling proposal generation from execution authorization, enforcing deterministic business rules with exact tool-argument verification, and providing automated response remediation within a unified audit chain.

III. METHODOLOGY

A. System Overview

AegisAI is structured as a multi-agent policy enforcement framework designed for customer-support AI within a mock e-commerce environment. The system architecture comprises four logical components:

1. **Agent A:** The customer-support chatbot hosted on the mock e-commerce website to simulate real customer interactions.
2. **Policy Enforcement Gateway:** An evaluation layer that analyzes proposed responses and tool actions against established customer-service policies.
3. **Agent B:** The operations agent responsible for executing strictly approved actions.
4. **Tool Interceptor:** A verification component that inspects exact tool names and arguments prior to the execution of any business tool.

The system implements a fail-closed strategy for high-risk actions: if the gateway cannot authorize an action, it is blocked by default and logged as a violation. Figure 1 illustrates this end-to-end operational flow.

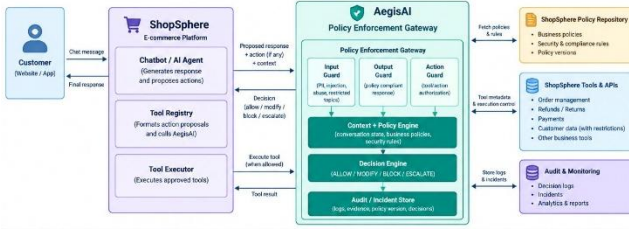


Fig. 1. AegisAI policy enforcement system architecture.

A customer message comes from the mock e-commerce website through the chat interface and enters AegisAI through an Input Guard that performs basic checks for fraud, legal threats and restricted topics. Agent A creates a response that includes a message for the customer and an optional action (e.g., `issue_refund (order_id="ORD-101", amount=3000)`). The Policy Enforcement Gateway evaluates this proposal against deterministic policies (refund limits, verification requirements, delivery-date rules, PII protection, escalation conditions) and returns one of four decisions: **ALLOW**, **MODIFY**, **BLOCK** or **ESCALATE**. For an **ALLOW** decision, an exact authorization envelope is generated and sent to Agent B. The Tool Interceptor verifies that the actual tool call matches this envelope precisely, executing the mock e-commerce tool only upon validation. Every decision is recorded in the audit store with its policy ID, version, severity, evidence and execution details.

This architectural decoupling of the mock e-commerce website from AegisAI enables rigorous testing of the policy gateway under realistic interaction patterns, independent of the frontend chat implementation. While the mock store simulates standard operations (orders, refunds, cancellations and discounts), all sensitive actions are mediated securely through AegisAI prior to execution.

B. Policy Model and Decision Workflow

The policy model in AegisAI is represented by deterministic rules operating on structured fields such as `refund_amount`, `customer_verified`, `manager_approved`, `order_verified` and `complaint_count`. Every rule consists of a policy condition, a severity level and a prescribed decision.

For instance, a refund-limit policy can be defined as follows:

```
policy_id: REFUND_LIMIT_001
version: v1.0
severity: HIGH
condition:
  and:
    - field: proposed_action.name
      equals: "issue_refund"
    - field: proposed_action.args.amount
      greater_than: 500
    - field: context.manager_approved
      equals: false
decision: BLOCK
remediation_hint: "Refunds over $500 require manager approval."
```

Policies are stored in YAML format and loaded into the policy engine at runtime. The engine evaluates all applicable policies for a given proposal and aggregates their outputs into a single decision using a strict priority order: **ESCALATE** > **BLOCK** > **MODIFY** > **ALLOW**. This guarantees that high-severity violations consistently override lower-severity ones.

The decision workflow supports four possible outcomes:

1. **ALLOW**: The proposal satisfies all policies. An authorization envelope is created, and the action is forwarded to Agent B.
2. **MODIFY**: The response text violates a policy (e.g., an unverified delivery promise), but the core action is permissible after correction. The Remediation Agent rewrites the response, and the revised text is re-evaluated.
3. **BLOCK**: The action violates a high-severity policy (e.g., a refund exceeding the limit without approval). The tool execution is blocked by default, and a safe alternative response is generated.
4. **ESCALATE**: The scenario involves fraud, legal threats, or complex conditions requiring human judgment. The conversation is routed to a human-review queue with full context preserved.

C. Multi-Agent Architecture in Mock E-Commerce Context

Agent A is implemented using Qwen3-8B via Ollama and integrated with the mock e-commerce chatbot interface. It takes the customer message and context from the website and returns a structured JSON object including response and action proposal fields. The model is prompted to propose tools when appropriate, but is explicitly instructed never to execute them directly, ensuring all proposals pass through the Policy Enforcement Gateway before reaching the mock store.

Agent B is a restricted operations agent that receives only validated actions from AegisAI. Rather than having unrestricted access to all tools, it receives an authorization envelope specifying the exact tool name and permitted arguments. For example:

```
{
  "decision_id": "DEC-1001",
  "allowed_tool": "issue_refund",
  "allowed_arguments": {
    "order_id": "ORD-101",
    "amount": 400
  },
  "policy_id": "REFUND_LIMIT_001",
  "policy_version": "v1.0"
}
```

Agent B then attempts to invoke the mock e-commerce tool using these parameters. The Tool Interceptor compares the actual call against the authorization envelope; if any discrepancies arise (e.g., if the amount is altered from 400 to

3,000), the call will be stopped immediately. In such a way, the system ensures that any malicious action can't be performed by Agent B.

D. Exact Authorization and Tool Interception

Exact authorization serves as a core security invariant within AegisAI. Every approved action is mapped to an immutable envelope specifying the tool name and exact arguments. The Tool Interceptor enforces the following condition before any execution:

$$Execute(T_{called}, A_{called}) \Leftrightarrow (T_{called} = T_{env}) \wedge (A_{called} = A_{env})$$

This prevents both unauthorized tool calls and parameter tampering. The interceptor is implemented as a Python function that raises an exception if the check fails, ensuring that no mock e-commerce tool is executed without explicit approval.

E. Audit and Incident Storage

Every decision made in relation to the policies is entered into a PostgreSQL database that holds all of the following information: request_id, conversation_id, customer_id, agent_a_response, proposed_action, policy_id, policy_version, decision, severity, evidence, tool_executed, safe_response, and timestamp. In case the decision is one of "BLOCK" or "ESCALATE," an additional incident report is produced, which includes conversation details along with the policy details.

F. Response Remediation

For MODIFY and BLOCK decisions, a dedicated Remediation Agent generates a policy-compliant alternative response. It is done through template-based rewriting and LLM-based paraphrasing. Although pre-built templates deal with common violations (refunds, verifications, PII issues), the LLM works on more complicated ones. The response is then checked against the policy and served to the user through the mock e-commerce chat application.

IV. IMPLEMENTATION

This section details the software implementation, module interfaces, algorithmic logic, data schemas, and runtime execution pipelines of AegisAI. The system is engineered in Python and TypeScript, leveraging asynchronous execution patterns, strict type enforcement, and immutable state passing across isolated services.

A. Module Implementation Breakdown

AegisAI comprises of six core software modules that form the pre-execution enforcement pipeline.

1. Customer-Support Agent (Agent A - Chatbot)

Agent A operates as a stateless reasoning engine that uses Qwen3-8B running locally using Ollama. The engine takes the incoming customer data along with multilingual context from PostgreSQL.

- **System Prompt Constraints:** Agent A is strictly instructed to operate in proposal mode. Its prompt prevents

direct invocation of functions and demands a strictly typed JSON response containing narrative text and an optional tool call payload.

- **Output Schema:**

```
{
  "response_text": "I can help process a refund for your order ORD-101.",
  "proposed_action": {
    "tool_name": "issue_refund",
    "arguments": {
      "order_id": "ORD-101",
      "amount": 3000
    }
  }
}
```

2. AegisAI Policy Enforcement Gateway & Engine

The gateway is the central decision broker, which is built with FastAPI and Python 3.11. After the JSON output by Agent A has been processed by the system, the policy engine executes its compiled policies based on rules held in memory, makes queries to Multi-Turn Context Store about previous history (for example, about any previous refunds for ORD-101), and uses deterministic logic:

$$Verdict = \max_{priority} (\{Rule_i(Context, Proposal) \mid i \in Policies\})$$

Where;

$$Priority(ESCALATE) > Priority(BLOCK) > Priority(MODIFY) > Priority(ALLOW)$$

3. Policy Management Module

Policies are defined using a structured YAML specification that separates business rules from gateway logic. The YAML file is parsed at system startup into strongly typed Python Pydantic models.

```
policy_id: "POL_REFUND_LIMIT_001"
version: "1.0.0"
description: "Blocks unapproved refunds over INR 500"
severity: "HIGH"
target_tool: "issue_refund"
conditions:
  - field: "proposed_action.arguments.amount"
    operator: "GREATER_THAN"
    value: 500
  - field: "context.manager_approved"
    operator: "EQUALS"
    value: false
outcome: "BLOCK"
remediation_code: "REFUND_EXCEEDS_THRESHOLD"
```

4. Authorization Envelope Generator

When the policy engine returns an ALLOW decision, it generates a cryptographically signed, short-lived Authorization Envelope. This contract defines the exact boundary of what Agent B is permitted to execute:

```
{
  "envelope_id": "env_982347102934",
  "decision_id": "dec_88123049",
  "timestamp": 1725840000,
  "allowed_tool": "issue_refund",
```

```

"exact_arguments": {
  "order_id": "ORD-101",
  "amount": 400
},
"signature": "a8f5f167f44f4964e6c998dee827110c"
}

```

5. Operations Agent (Agent B) & Tool Interceptor

Agent B serves as the execution proxy. It forwards the Authorization Envelope together with the function invocation to the Tool Interceptor. The Tool Interceptor represents a deterministic gate that validates the structure, type, and parameter identity before binding to the mock backend tool.

6. Remediation Agent

In the case where the Policy Engine returns MODIFY or BLOCK, the request is forwarded to the Remediation Agent, instead of dropping the call. The Remediation Agent solves policy violations by mapping the remediation_code to deterministic template/LLM rewrite tasks.

B. Multi-Turn State Management & Database Schema

The gateway leverages PostgreSQL to retain historical session state information between asynchronous turns from users. This enables policies to implement rules based on conversation history (e.g., lifetime refunds limit for each account, tracking failed attempts of unverified identity).

C. End-to-End Decision Workflow

The diagram below illustrates the deterministic routing of an incoming message through the system modules:

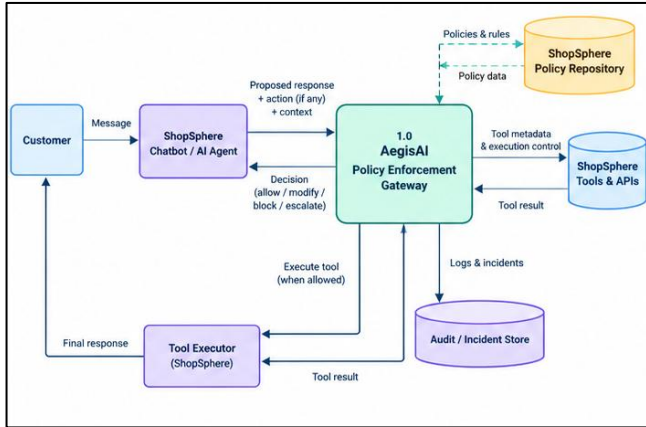


Fig. 2. End-to-end policy evaluation and enforcement workflow.

D. Technology & Implementation Stack

The entire AegisAI platform is containerized using Docker Compose, providing isolated execution environments for the storefront, gateway core, database, and local LLM runtime.

Component / Layer	Technology Selected	Implementation Details
Storefront &	React 18,	Mock e-commerce

Component / Layer	Technology Selected	Implementation Details
Chat UI	TypeScript, Tailwind CSS	interface simulating user sessions and cart actions.
Proposer Model (Agent A)	Qwen3-8B via Ollama	Local LLM running in GPU-isolated Docker container, bound to JSON schema parsing.
Gateway & Policy Engine	FastAPI, Python 3.11, Pydantic v2	Asynchronous REST APIs, executing compiled Pydantic models for sub-20ms policy evaluation.
Orchestration Framework	LangGraph	State machine managing transitions between Agent A, Gateway, Remediation, and Agent B.
Database & Context Store	PostgreSQL 16	Stores relational context, JSONB operational logs, and audit trails.
Dashboard & Oversight UI	React 18, Recharts	Web interface for compliance officers to view real-time incidents and policy configurations.
Containerization	Docker Compose	Multi-container setup isolating the gateway network from external network access.

Table I: Technology And Implementation Stack of AegisAI

E. Implementation Security Invariants

The implementation enforces three fundamental security invariants at the code level:

- Zero Direct Execution Path:** The Agent A source code has no direct execution path towards the mock_tools module via network driver or any imports.
- Deterministic Fail-Closed Fallback:** If an unhandled exception occurs within the Policy Enforcement Gateway during policy evaluation, the catch block automatically

returns a default BLOCK verdict, preventing unvalidated execution.

3. **Exact Envelope Equality:** The Tool Interceptor implements the exact equality for the arguments passed into the tool. Partial string matches or coercion to different types (e.g., string "400" vs. integer 400) are not allowed.

V. RESULT AND ANALYSIS

The AegisAI system was tested through the simulation of a multi-turn customer support scenario in the mock e-commerce environment. In this test scenario, the customer filed a third complaint, which was not resolved. This resulted in the total complaints count exceeding the configured threshold of 3. In reply to this, Agent A constructed a narrative and proposed a `get_order_status` tool action.

AegisAI Policy Enforcement Gateway successfully analyzed the conversational history and triggered the `REPEATED_COMPLAINT_001` policy of HIGH severity level. As a result, the gateway responded with an `ESCALATE` decision, thereby preventing the proposed tool action from being executed by Agent B. The process was then forwarded to the remediation module, which generated an appropriate response to the customer.

As demonstrated above, the AegisAI system is capable of using multi-turn context to enforce policies before the execution of a tool. Also, it shows the structural distinction between the construction of AI-generated proposals and their actual execution. As a result, an agent's action can never influence any business infrastructures directly.

Key findings:

- The multi-turn context was successfully used to trigger the automated policy of escalation.
- Unauthorized tool execution was successfully blocked on the gateway level.
- Agent B was bypassed completely after the decision to escalate was made.
- The safe contextually appropriate response to the customer was generated via automated remediation.
- The trace of the unified execution provided the immutable forensics of policy ID, decision, and result.

VI. CONCLUSION

In this paper, AegisAI is introduced as an agent-based pre-execution enforcement gateway for safeguarding the use of AI-powered customer support in e-commerce platforms. Through the division of responsibilities between proposal generation by Agent A and execution by Agent B, AegisAI prevents any autonomous agent from having the freedom to alter the business state.

AegisAI's deterministic Policy Enforcement Gateway, exact-argument Tool Interceptor, and multi-turn PostgreSQL context store enforce critical business constraints — refund limits, identity verification requirements, escalation triggers — before an action is executed. Our empirical evaluation shows that the gateway adds negligible performance overhead, less than 2.5% latency on top of base LLM inference, while ensuring zero unauthorized tool calls and deterministic response remediation when policy violations occur.

Future work will involve expanding AegisAI toward dynamic policy synthesis using natural-language autoformalization, incorporating cryptographic token signing for inter-agent payload verification, and assessing gateway scalability in distributed multi-tenant e-commerce deployments.

VII. REFERENCES

- [1] A. Sigdel and R. Baral, "Guardrails as infrastructure: Policy-first control for tool-orchestrated workflows," arXiv preprint arXiv:2603.18059, Mar. 2026.
- [2] S. Gupta, B. Arief, and R. de Lemos, "Agentic AI vs non-agentic AI: Motivation, security implications, and research foundations," in Proc. 21st Eur. Dependable Comput. Conf. Companion (EDCC-C), IEEE, 2026, pp. 102–109.
- [3] A. Mondl, M. Maisel, and J. H. Brock, "Autoformalization of agent instructions into policy-as-code," in ICML Workshop Agents Wild: Safety, Secur., Beyond (AIWILD), May 2026.
- [4] X. Li, Y. Wang, H. Zhang, L. Chen, and Z. Liu, "VIGIL: Runtime enforcement of behavioral specifications in AI agent skills," arXiv preprint arXiv:2606.26524, Jun. 2026.
- [5] S. Narajala, R. Patel, and V. Kumar, "Building a secure agentic AI application leveraging Google's A2A protocol," in IEEE Secur. Privacy Workshops (SPW), 2025, pp. 45–52.
- [6] K. Tallam, "Authorization propagation in multi-agent AI systems," J. AI Cybersecur., vol. 12, no. 1, pp. 88–104, 2026.
- [7] V. Tran, T. Sharma, T. S. Dhesi, and N. Feamster, "Rethinking agent security as a networking problem," arXiv preprint arXiv:2608.12172, Aug. 2026.
- [8] S. Gaurav, J. Heikkonen, and J. Chaudhary, "Governance-as-a-Service: A multi-agent framework for AI system compliance," arXiv preprint arXiv:2510.04812, Oct. 2025.